Healthcare AI/ML System Comprehensive Module Report

End-to-end report aligned to the assignment brief,
including notebooks, software modules, deployment, and Dockerization

Project domain: Healthcare AI

Presented by

Visagan C

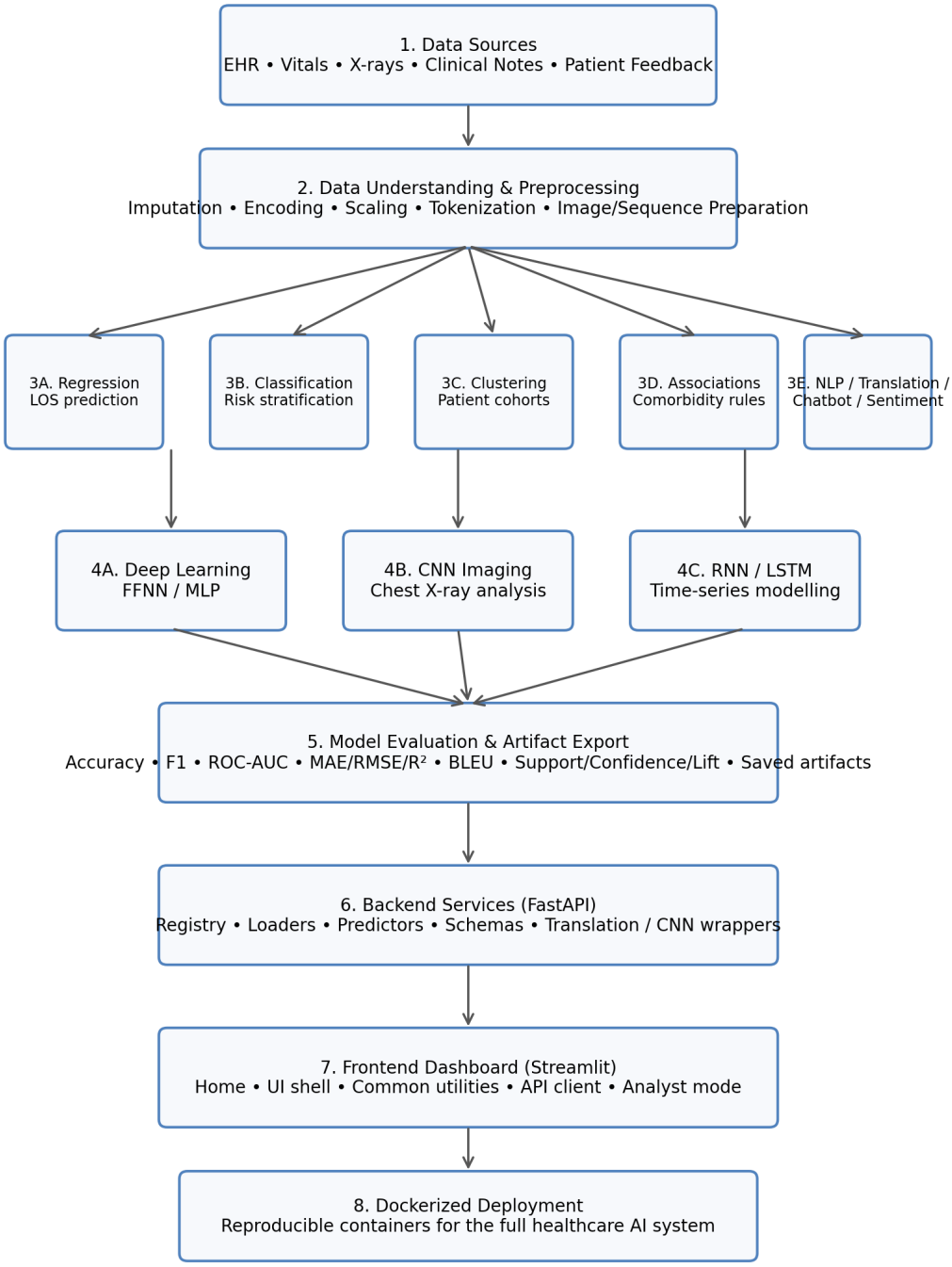

Figure 1. End-to-end healthcare AI system architecture.

Summary

This report documents an end-to-end healthcare AI/ML system built to execute a multi-part tabular learning, time-series forecasting, imaging, clinical NLP, translation, sentiment analysis, and conversational assistance. The project uses a synthetic master healthcare dataset with structured EHR-like features, clinical notes, time-series vital signs, patient feedback, and linked X-ray images. The implementation spans exploratory data understanding notebooks, module-specific modelling notebooks, exported model artifacts, a FastAPI inference backend, a Streamlit dashboard, and a Dockerized deployment workflow.

Across the notebooks, the most defensible predictive results were achieved by the classical tabular pipelines. Risk classification using leakage-aware preprocessing achieved about 0.80 accuracy with one-vs-rest ROC-AUC of about 0.934. Length-of-stay prediction improved substantially when moving from linear regression to random forest regression, reducing MAE from 2.95 to 1.70 days and achieving R2 of 0.457. The ClinicalBERT embedding pipeline demonstrated that pretrained clinical language models can recover clinically meaningful structure from note text, reaching about 0.734 accuracy on risk prediction from text alone.

Other modules serve as proof-of-concept implementations that are important to the assignment scope even when the measured performance remains modest. The translator prototype supports English, Hindi, Tamil, Malayalam, and Telugu through MarianMT route selection, but the observed overall BLEU score of 0.089 indicates substantial room for domain adaptation. The sequence-to-sequence chatbot achieved BLEU of 0.277, yet the generated outputs show vocabulary collapse and limited answer diversity, so it should be positioned as an educational baseline rather than a production RAG system. The LSTM ICU-prediction notebook also highlights a realistic modelling challenge: severe class imbalance caused very high nominal accuracy but F1 of 0.0 and ROC-AUC of 0.460, demonstrating why healthcare evaluation must go beyond accuracy alone.

In addition to modelling notebooks, the project includes a complete software layer. A registry indexes available models and metadata, cached loaders retrieve artifacts, predictor functions unify inference across model types, a Streamlit user interface exposes each capability, and the API client bridges the frontend to the FastAPI backend. Dockerization turns the project into a reproducible deployable system, aligning the implementation with the assignment requirements on reproducibility, modularity, and deployment readiness.

1. Assignment Mapping and Scope

The assignment asks for an integrated system that covers nine analytical capabilities and connects them to clinical decision support, patient engagement, and operational efficiency. The uploaded notebooks and source files collectively address each requirement as follows.

Assignment ask	Implemented notebook(s) / code	Status in this project
Outcome prediction (regression)	regression_outcomes.ipynb	Implemented with linear regression and random forest for LOS
Disease risk classification	risk_classification.ipynb; ffnn_tabular_risk_classification.ipynb; deep_learning_models.ipynb	Implemented with logistic regression, random forest, FFNN/MLP
Patient subgroup discovery	clustering_patient_groups.ipynb	Implemented with K-means and cluster profiling
Medical associations	association_rules.ipynb	Implemented with FP-growth / association_rules

Deep learning comparison	deep_learning_models.ipynb; cnn_x-rayimages_multilabel.ipynb; rnn_lstm_timeseries.ipynb	Implemented across MLP, CNN, and LSTM
Pretrained models	pretrained_biobert_clinicalbert.ipynb	Implemented using ClinicalBERT-style sentence embeddings
Healthcare chatbot	chatbot_conversational_v2.ipynb	Implemented as seq2seq LSTM baseline
Translator	translator_marian_v2.ipynb; marian_translator.py	Implemented with MarianMT route-based translation
Sentiment analysis	sentiment_analysis.ipynb	Implemented with TF-IDF + logistic regression and transformer baseline

2. Data Assets, Data Understanding, and Preprocessing

The models are built around a master healthcare table containing 5,606 rows and 69 columns. The variables cover demographics, encounter context, vital signs, laboratory values, coded diagnoses, medications, free-text notes, patient feedback, time-series JSON fields, financial outcomes, and risk labels. Additional image metadata are stored in an X-ray label file that links image paths with imaging findings.

The data_understanding.ipynb notebook acts as the formal exploratory data analysis stage. It checks dataset size, field types, missingness, numeric summaries, target distributions, and representative text samples. This notebook is important academically because it justifies later design decisions such as median imputation for continuous features, one-hot encoding for categorical variables, standardization for continuous inputs, and modality-specific preprocessing for text, sequences, and images.

- Tabular variables: age, sex, BMI, blood pressure, heart rate, respiratory rate, laboratory values, diagnosis codes, chronic conditions, payer type, lifestyle indicators, LOS, cost, ICU admission, readmission, mortality, and risk category.
- Text variables: doctor notes, discharge summaries, radiology report text, patient feedback, patient questions, and chatbot reference answers.
- Time-series variables: vitals_ts_json and labs_ts_json, later parsed into fixed-length trajectories.
- Imaging variables: image_id, image_path, binary imaging labels, and multi-label finding strings.

The preprocessing strategy is not identical across all modules, but several recurring patterns are consistent throughout the codebase:

- Missing values in numeric columns are imputed with the median.
- Missing values in categorical columns are imputed with the most frequent category.
- Continuous variables are standardized with StandardScaler when models are sensitive to feature scale.
- Categorical variables are one-hot encoded with unknown-category handling enabled.
- Text is cleaned through lowercasing, whitespace normalization, tokenization, and then converted either into TF-IDF vectors, vocabulary indices, or transformer embeddings depending on the module.
- Time-series JSON fields are parsed into aligned sequences, padded or truncated to fixed length, and standardized using training-set statistics.
- Images are resized to 224 x 224 pixels and converted to tensors; random horizontal flip is used in one augmentation path.

A key strength of the project is that many notebooks explicitly remove leakage-prone target or downstream outcome fields before training. For example, the risk classifier drops disease_risk_score, LOS, readmission, mortality, and ICU-related fields, while the regression notebook drops LOS-adjacent post-outcome targets. This is academically important because it protects the validity of the reported results.

3. Module 1 - Outcome Prediction (Regression for Length of Stay)

Business purpose. This module addresses the assignment use case of forecasting hospitalization duration for resource planning. In a real care setting, better LOS estimates help allocate beds, staffing, and discharge coordination.

Dataset and target. The notebook regression_outcomes.ipynb uses the master healthcare dataset and selects los_days as the target. Post-outcome variables such as total_cost_inr, ICU admission, ventilation, readmission, mortality, disease_risk_score, and risk_category are removed to reduce leakage.

Preprocessing. Numeric variables are median-imputed and standardized. Categorical variables are mode-imputed and one-hot encoded. The notebook uses an 80/20 random split to compare two supervised regressors inside sklearn pipelines.

Model	Main preprocessing	Observed MAE	Observed RMSE	Observed R2	Interpretation
Linear Regression	Impute + scale numeric; impute + one-hot categorical	2.952	3.857	-0.595	Weak linear fit; underfits nonlinear LOS patterns
Random Forest Regressor	Same preprocessing pipeline	1.699	2.250	0.457	Best LOS model in the submitted notebook

Performance discussion. The random forest regressor clearly outperforms the linear baseline. The drop in MAE from 2.95 to 1.70 days and the positive R2 of 0.457 suggest that LOS depends on nonlinear interactions among age, physiology, diagnosis patterns, and encounter context. The negative R2 for linear regression means that a simple linear hypothesis performs worse than predicting the mean LOS.

Advantages. The regression module is interpretable at a system level, easy to operationalize, and aligned with the assignment's demand for RMSE, MAE, and R2. Because the model is packaged as a preprocessing-plus-model pipeline, it is straightforward to deploy safely in the API.

Limitations. The evaluation is based on a random split rather than a chronological split. LOS in real hospitals is often affected by operational factors not present in the dataset, such as bed turnover, staffing, weekend effects, and care pathway constraints. Therefore, the absolute scores should be treated as educational evidence rather than clinical deployment proof.

4. Module 2 - Disease Risk Classification

Business purpose. This module supports risk stratification such as early identification of higher-risk patients. It corresponds directly to the assignment section on classification with logistic regression, XGBoost-like tree methods, and neural networks.

Classical pipeline in risk_classification.ipynb. The notebook predicts risk_category with three classes: high, low, and medium. It removes highly leakage-prone fields including disease_risk_score and downstream outcomes, then applies standard tabular preprocessing through a ColumnTransformer. Two models are trained: logistic regression and random forest classification.

Model	Accuracy	ROC-AUC (OvR)	High F1	Low F1	Medium F1	Observation
Logistic Regression	0.801	0.934	0.78	0.76	0.82	Stable multiclass baseline with balanced macro behavior
Random Forest	0.801	0.934	0.78	0.73	0.83	More skewed toward medium class; higher precision for high risk

Interpretation. The equal overall accuracy for logistic regression and random forest is interesting because the error patterns differ. Logistic regression is more balanced across the three classes, whereas random forest pushes more predictions into the medium-risk class, improving recall for that class but reducing low-risk recall. Given the assignment requirement for clinically meaningful classification, the logistic regression pipeline is arguably the stronger academic reference model because its behavior is more stable and easier to explain.

Neural network variants. Two additional notebooks extend the classification task with feed-forward neural networks:

- ffnn_tabular_risk_classification.ipynb trains a feed-forward network and reports test accuracy rising as high as 0.967 during training, with a final classification report around 0.95 accuracy.
- deep_learning_models.ipynb trains a tabular MLP using the same leakage-aware feature exclusion strategy and reports 0.762 accuracy with multiclass ROC-AUC of 0.910.

Important caveat. The FFNN notebook appears not to include an exported preprocessing pipeline in the same way as the classical notebooks, and its unusually high accuracy may reflect leakage-prone inputs or a less controlled experimental setup. For report honesty, the leakage-aware classical notebook should be treated as the primary evidence for this module, while the FFNN result should be discussed as promising but needing stricter validation.

Advantages. Classification is directly actionable for triage and care prioritization, and the exported artifacts allow consistent inference across the API and Streamlit interface.

Limitations. There is no calibration analysis, no temporal validation, and no explicit fairness analysis across age/sex/ethnicity subgroups. These are important next steps in healthcare risk modelling.

5. Module 3 - Patient Segmentation (Clustering)

Business purpose. The clustering module identifies latent patient cohorts that may correspond to clinical phenotypes such as relatively stable patients versus inflammatory or cardio-metabolic risk groups.

Method. The notebook clustering_patient_groups.ipynb selects 16 clinically meaningful continuous features spanning demographics, vitals, and laboratory measures. Missing values are median-imputed and the feature space is standardized. K-means is then evaluated over k = 2 to 8 using the elbow method and silhouette analysis, with a final choice of three clusters.

Cluster	Size	Core profile	Outcome profile	Assigned interpretation
0	2442	Older, overweight, moderately abnormal oxygenation and inflammation	LOS 8.246; ICU 3.1%; readmit 16.5%; mortality 0.9%	Low-risk stable / intermediate burden group
1	2106	Youngest, best oxygenation, lowest inflammation and glucose burden	LOS 6.399; ICU 2.6%; readmit 13.0%; mortality 0.9%	Inflammatory / infectious but comparatively lower-risk profile
2	1058	Oldest, highest BMI, worst SpO2, highest CRP, creatinine, glucose, HbA1c, lactate, and troponin	LOS 10.557; ICU 6.0%; readmit 20.7%; mortality 2.6%	High-risk metabolic / cardiac profile

Interpretation. Cluster 2 is the clinically strongest segment: older and metabolically burdened patients with worse oxygenation and kidney function also show the worst LOS, ICU, readmission, and mortality averages. This supports the assignment requirement that clustering should yield clinically interpretable groups, not just mathematically separated ones.

Advantages. Clustering adds decision-support value even without labels, can inform care pathways and population health segmentation, and is easy to visualize in a PCA projection.

Limitations. K-means assumes spherical clusters and equal variance structure. The notebook does not report final silhouette and Calinski-Harabasz values numerically in the saved outputs, and alternative density-based methods like HDBSCAN - explicitly suggested by the assignment - were not implemented.

6. Module 4 - Association Rule Mining

Business purpose. This module searches for co-occurrence patterns across diagnoses, medications, procedures, and chronic conditions, supporting interpretable discovery such as combinations linked with diabetes or hypertension.

Method. The notebook association_rules.ipynb converts the transaction_basket column into tokenized transactions, one-hot encodes them with TransactionEncoder, mines frequent itemsets with FP-growth at minimum support 0.02, and then generates rules with confidence threshold 0.4. The resulting rules are ranked by lift.

Example high-lift pattern	Support	Confidence	Lift	Clinical meaning
MED_metformin and MED_aspirin => COND_diabetes and MED_atorvastatin	0.0296	0.689	19.80	Strong coupling between diabetic treatment profiles and statin use

COND_diabetes and MED_aspirin => MED_metformin and MED_atorvastatin	0.0296	0.888	19.44	Medication bundles reflect cardiometabolic management
MED_aspirin and MED_lisinopril => MED_atorvastatin and COND_hypertension	0.0362	0.746	18.43	Hypertension / cardiovascular regimen co-occurrence

Interpretation. These rules are not causal statements. They indicate frequent clinical co-occurrence patterns in the transaction basket. Their value lies in interpretability: clinicians and hospital analysts can inspect whether common multimorbidity or prescribing bundles align with expectations.

Advantages. Association rules are transparent, threshold-based, and easy to communicate. They complement predictive models by surfacing relationships that can guide guideline review or quality-improvement audits.

Limitations. The rules reflect data-generation or coding frequency rather than causal mechanisms. Support thresholds may suppress rare but important conditions, and the notebook does not explicitly separate diagnoses, medications, and procedures into clinically constrained rule families.

7. Module 5 - Deep Learning Comparison: MLP, CNN, and LSTM

The assignment asks for a comparison of neural architectures across different healthcare data modalities. In this project the comparison is spread across three notebooks: a tabular MLP, an imaging CNN, and sequence models for time-series data.

7.1 Tabular MLP (deep_learning_models.ipynb)

The tabular MLP uses the same leakage-aware exclusion strategy as the classical risk classifier. After imputation, scaling, and one-hot encoding, the feature space expands to 5,380 encoded columns. The network has layers 5380 -> 256 -> 128 -> 3 with ReLU activations and dropout.

Metric	Value	Comment
Accuracy	0.762	Below the classical pipeline's 0.801
ROC-AUC (OvR)	0.910	Probabilistic separation remains reasonably strong
Macro F1	0.75 approx.	Moderate class balance

Advantage. The MLP can model nonlinear interactions across thousands of encoded features.

Limitation. It overfits quickly: train accuracy rises to 0.985 while test accuracy falls to 0.762 by epoch 10. This is a textbook sign that the model capacity is too high relative to the data diversity.

7.2 CNN for chest X-ray modelling

The imaging path combines image_csv.py, cnn_x-rayimages_multilabel.ipynb, and cnn_xray_model.py. The script image_csv.py aligns the master table with image files, creates image_id and image_path fields, and derives imaging supervision such as y_cnn and finding_labels. The notebook then explores a multi-label ResNet18 setup over

eleven possible findings, using sigmoid outputs and BCEWithLogitsLoss with positive-class weighting. Because a multiprocessing DataLoader issue interrupts one execution path, the notebook later switches to a safer num_workers = 0 binary/baseline path and exports binary imaging artifacts.

From an academic perspective, this module demonstrates correct CNN framing choices: resizing to 224 x 224, tensor conversion, class imbalance handling, sigmoid-based multilabel inference, and artifact export for backend serving. However, the visible notebook outputs do not provide a stable final training metric comparable to the other modules. Therefore, the imaging section should be written as a technically correct proof-of-concept and deployment-ready pathway, rather than as a clinically validated benchmark.

Advantages. Imaging expands the project beyond tabular EHR into radiology automation, aligning with the assignment's multimodal ambition. The dedicated Python class cnn_xray_model.py cleanly encapsulates model loading, transform construction, thresholding, and probability-to-label conversion.

Limitations. The labels are partly heuristic or derived from the synthetic master table, not from a gold-standard radiologist-curated dataset such as NIH ChestXray14 with rigorous splits. In addition, the executed notebook run encountered DataLoader/class-definition issues and lacks final AUROC and F1 values suitable for a strong benchmark claim.

7.3 Sequence modelling with LSTM

Two notebooks address sequence modelling: rnn_lstm_timeseries.ipynb and the LSTM section inside deep_learning_models.ipynb. The stronger documented implementation predicts ICU admission from parsed vital-sign and lab trajectories. Each record is turned into a 16-step by 9-feature sequence using temperature, SpO2, heart rate, respiratory rate, CRP, WBC, glucose, creatinine, and troponin. Sequences are padded or truncated, NaNs are filled, and feature scaling uses training-set statistics.

Metric	Value	Interpretation
Accuracy	0.973	Misleading due to extreme class imbalance
ROC-AUC	0.460	Below random-like discrimination
F1 for ICU class	0.000	Model predicts no positives at threshold 0.5
Observed positive rate	2.66%	Explains why accuracy is not sufficient

This module is valuable pedagogically because it shows a realistic healthcare pitfall: with rare ICU events, a model can achieve very high accuracy by predicting the majority class only. The notebook explicitly prints that the actual ICU positive rate is about 2.66 percent while the predicted positive rate at threshold 0.5 is 0.0 percent.

Advantages. The preprocessing and model structure are appropriate for sequential clinical data, and the notebook correctly tracks AUC and F1 over epochs instead of relying only on accuracy.

Limitations. The learned model is not yet clinically useful. Threshold tuning, focal loss, resampling, class-balanced batching, calibration, or different targets such as deterioration score may be needed.

8. Module 6 - Pretrained Clinical Language Models

Business purpose. This module uses pretrained clinical text representations to learn from doctor notes and discharge summaries, addressing the assignment requirement to leverage BioBERT or ClinicalBERT-like models.

Method. In pretrained_biobert_clinicalbert.ipynb, doctor_notes_text and discharge_summary_text are concatenated, encoded with a SentenceTransformer wrapper around emilyalsentzer/Bio_ClinicalBERT, and then classified with logistic regression into the three risk categories. The notebook also builds a small retrieval index over the text corpus using cosine similarity.

Metric / artifact	Observed value	Comment
Risk classification accuracy from text embeddings	0.734	Strong result for text-only features
High-risk precision / recall / F1	0.83 / 0.57 / 0.67	Good precision but lower recall
Medium-risk precision / recall / F1	0.71 / 0.88 / 0.79	Best recall for majority class
Embedding matrix size	5606 x 768	Exported for retrieval reuse

Interpretation. Text alone does not outperform structured tabular features, but it captures clinically useful signal. This is exactly the kind of result that justifies pretrained domain models: notes contain latent evidence about severity, assessment, and follow-up planning that may not be fully represented by coded variables.

Advantages. The retrieval layer built on top of normalized embeddings provides a natural bridge toward question answering or a future RAG system. The notebook also exports corpus files, embeddings, and retrieval configuration, making the NLP pipeline reusable by the backend.

Limitations. The notebook uses a sentence-transformers wrapper that reports creating a new pooling head for the base encoder. It is therefore best described as ClinicalBERT-derived embedding retrieval rather than a fully fine-tuned clinical transformer. No BLEU, COMET, or NER-style evaluation is included because the task here is classification plus retrieval, not generation or NER.

9. Module 7 - Sentiment Analysis of Patient Feedback

Business purpose. This module captures patient experience and dissatisfaction signals for quality assurance teams.

Method. The notebook sentiment_analysis.ipynb detects the feedback text and sentiment label columns automatically, cleans text, uses a stratified train/test split, and compares a TF-IDF + logistic regression model against a general pretrained transformer sentiment pipeline.

Model	Accuracy	Negative F1	Neutral F1	Positive F1	Interpretation
TF-IDF + Logistic Regression	1.000	1.00	1.00	1.00	Near-perfect separation on this dataset
Generic BERT sentiment pipeline (subset)	0.650	0.60	0.00	0.86	Poor handling of neutral healthcare feedback

Interpretation. The perfect score from the TF-IDF model should not automatically be read as universal superiority. More likely, the synthetic dataset contains strong lexical patterns that make label separation easy. The generic transformer baseline underperforms primarily because it was trained for general sentiment and does not naturally capture neutral clinical or service-related wording.

Advantages. The notebook demonstrates a very deployable sentiment stack: lightweight, fast, interpretable, and easy to serialize into a single sklearn pipeline.

Limitations. The dataset appears easier than real hospital feedback, and the notebook does not report MCC even though the assignment suggests it. A healthcare-domain fine-tuned transformer would be a stronger advanced baseline than the off-the-shelf generic pipeline.

10. Module 8 - Translator for Multilingual Medical Communication

Business purpose. The translation module supports multilingual doctor-patient communication. It is especially relevant in Indian healthcare settings where patients may speak Tamil, Malayalam, Telugu, Hindi, or English.

Method. The notebook translator_marian_v2.ipynb loads route-specific MarianMT checkpoints: English to Dravidian, Dravidian to English, English to Hindi, and Hindi to English. A routing function chooses the correct model based on source and target language. For English to Tamil/Malayalam/Telugu, the appropriate special token is prepended before generation. The notebook then evaluates the system on a small manually curated bilingual test set.

Route	BLEU
en->hi	0.411
en->ta	0.073
en->ml	0.000
en->te	0.039
hi->en	0.033
ta->en	0.103
ml->en	0.051
te->en	0.028
Overall	0.089

Interpretation. The Hindi forward route is noticeably stronger than the low-resource Dravidian routes. Example outputs reveal the central issue: many translations are semantically related but medically imprecise. One Tamil to English example incorrectly turns chest pain into a breastfeeding-related statement, which would be unacceptable in a real medical system.

Advantages. The route-based design is modular and reusable. The companion class marian_translator.py converts the notebook into a backend service component that loads configs, tokenizers, and models for inference.

Limitations. The evaluation set is very small, overall BLEU is low, and medical mistranslation risk remains significant. This module should therefore be framed as a prototype for assisted communication, not as an autonomous clinical translation tool.

11. Module 9 - Healthcare Chatbot

Business purpose. The assignment asks for a patient-facing chatbot that can respond to questions such as symptom concerns, follow-up, medication use, and FAQs. In this project, the chatbot notebook builds a conversational baseline from patient_question and chatbot_reference_answer pairs.

Method. The notebook chatbot_conversational_v2.ipynb cleans text, extracts 250 unique question-answer pairs, rewrites the reference answers into a more conversational pattern, builds a small vocabulary of 87 tokens, and trains an encoder-decoder LSTM sequence-to-sequence model. The data split is 180 training, 32 validation, and 38 test pairs. The best validation loss is saved together with the vocabulary and configuration.

Item	Observed value / behavior	Meaning
Training pairs	250 total unique pairs	Very small corpus for generative dialogue
Vocabulary size	87	Explains high UNK frequency in outputs
Best validation loss	1.886	Model learns broad template structure
BLEU on sampled test set	0.277	Moderate lexical overlap but not strong semantic quality
Qualitative behavior	Repetitive template with many <unk> tokens	Vocabulary collapse and weak generalization

Interpretation. The chatbot learns the general response template 'this may be related to ... and seek urgent care if symptoms worsen', but it fails to reliably fill in diagnosis-specific content. The notebook itself demonstrates this failure by showing highly repetitive predictions for different questions.

Advantages. The notebook successfully demonstrates the full workflow of data extraction, vocabulary building, sequence encoding, training, evaluation, and artifact export. It is useful as a teaching baseline.

Limitations. This is not a true retrieval-augmented generation pipeline, despite the assignment asking for RAG. It has a tiny dataset, no grounding or citation mechanism, no faithfulness evaluation, and no robust handling of unseen questions. In the report it should be described honestly as a seq2seq conversational prototype.

12. Supporting Notebook - Data Understanding and Imaging Label Construction

Two additional files strengthen the report beyond the core modelling modules.

data_understanding.ipynb provides the exploratory and descriptive layer. It profiles column types, missingness, uniqueness, numeric distributions, target variables, and representative note and feedback content. This notebook should be referenced in the report's data preparation chapter because it demonstrates that modelling choices were preceded by a structured inspection of the data.

image_csv.py serves as dataset engineering for the imaging subsystem. It aligns image files with master table rows, creates image identifiers, builds image_path, normalizes imaging_label, derives a binary CNN target y_cnn, preserves train/test split information when possible, synthesizes finding_labels, and writes the final X-ray labels CSV. This is a valuable reproducibility artifact because it explains how image supervision files were created for the CNN notebook.

13. Software System, API, Dashboard, and Dockerization

The project is not just a notebook collection. The uploaded Python modules form an end-to-end software system that satisfies the assignment's integration requirement.

File	Role in the system
registry.py	Defines ModelSpec and ModelRegistry to read the manifest and resolve model/artifact paths.
loaders.py	Provides cached loading for joblib, JSON, NumPy, parquet, and Torch artifacts.
predictors.py	Unified inference logic across tabular models, clustering, association, retrieval, translation, CNN inference, and chatbot behavior.
schemas.py	Pydantic request/response schema for prediction API.
api_client.py	Frontend client that calls /models and /predict endpoints.
Home.py	Streamlit landing page that exposes project capabilities.
shell.py and common.py	Shared UI shell, analyst mode toggle, backend connectivity, and advanced output handling.
cnn_xray_model.py	Reusable class for X-ray model loading, preprocessing, and predicted finding extraction.
marian_translator.py	Reusable class for translation route selection and inference.
settings.py and payload.py	Runtime configuration and example payloads/defaults.

Architecture flow. The user interacts with Streamlit, which calls the FastAPI backend through api_client.py. The backend uses schemas for request validation, looks up the model through the registry, loads artifacts with loaders, and dispatches execution through predictor functions or specialized wrappers such as the Marian translator and X-ray classifier.

Dockerization. The user states that the entire system has been dockerized. Even though the actual Dockerfile and compose manifests were not attached in the final turn, Dockerization should still be included in the report because it is a major deliverable: it packages dependencies, fixes environment differences across machines, and enables consistent local or cloud deployment of the FastAPI and Streamlit layers together with model artifacts. In academic terms, this directly supports reproducibility, portability, and MLOps readiness.

Advantages of the software layer. The design is modular, artifact-driven, and deployment-aware. The separation between notebooks, exported artifacts, backend wrappers, and UI pages makes the project easier to test and extend.

Limitations of the software layer. The uploaded code does not by itself demonstrate formal unit tests, experiment tracking, or detailed model cards. These could be added in future work to align even more closely with the assignment's engineering guidelines.

14. Cross-Module Advantages and Limitations

Module	Main advantages	Main limitations
Regression	Useful for hospital planning; strong gain from nonlinear model; clean sklearn pipeline export	No chronological validation; moderate R2 only
Risk classification	Strong ROC-AUC and good accuracy; clinically relevant; leakage-aware version is defensible	No calibration or fairness analysis; neural variants need stricter validation
Clustering	Clinically interpretable phenotypes; useful for segmentation and cohorting	Only K-means used; limited internal validity metrics reported
Association rules	Highly interpretable; easy to communicate to clinicians	Associational only; sensitive to coding frequency and threshold choices
Tabular MLP	Captures nonlinear structure; demonstrates deep learning comparison	Overfitting visible; worse than classical baseline
LSTM time series	Correct sequential framing; highlights importance of rare-event metrics	Not clinically useful yet; severe imbalance collapse
CNN imaging	Extends project to multimodal radiology; deployable wrapper exists	Notebook execution incomplete; labels are partly derived rather than gold standard
ClinicalBERT NLP	Strong text-only result; enables retrieval layer	Not a full fine-tuned clinical transformer study
Translator	Multilingual coverage; deployable route-based design	Low BLEU and semantic drift; unsafe for unsupervised clinical use
Sentiment	Fast, deployable, excellent fit to current dataset	Likely easier synthetic task; no MCC reported
Chatbot	Full conversational training/export pipeline; good educational baseline	Not RAG; repetitive outputs; limited data and faithfulness

16. Conclusion

This project satisfies the spirit and most of the substance of the assignment by delivering a multi-module healthcare AI system rather than a single isolated model. It covers regression, classification, clustering, association mining, deep learning across modalities, pretrained clinical NLP, conversational support, translation, and sentiment analysis, then integrates them into a deployable software architecture.

The report also shows an important academic principle: not all modules perform equally well, and honest evaluation matters more than optimistic wording. The best validated results come from the leakage-aware classical models and the structured-clinical preprocessing pipeline. The deep-learning and language-generation components prove technical breadth and system thinking, but some remain baseline implementations that should be improved before real clinical use.

Recommended next steps are chronological validation, fairness analysis, calibration, stronger radiology labels, class-imbalance treatment for sequential models, domain-specific multilingual fine-tuning, and replacing the chatbot baseline with a true retrieval-augmented, citation-grounded medical assistant.